Chapter 9. Licenses, Copyrights, and Patents

The license you select probably won't have a major impact on the adoption of your project, as long as the license is open source. Users generally choose software based on quality and features, not on the details of the license. Nevertheless, you still need a basic understanding of free software licensing issues, both to ensure that the project's license is compatible with its goals, and to be able to discuss licensing decisions with other people. Please note, however, that I am not a lawyer, and that nothing in this chapter should be construed as formal legal advice. For that, you'll need to hire a lawyer or be one.

Terminology

In any discussion of open source licensing, the first thing that becomes apparent is that there seem to be many different words for the same thing: *free software*, *open source*, *FOSS*, *F/OSS*, and *FLOSS*. Let's start by sorting those out, along with a few other terms.

free software
: Software that can be freely shared and modified, including in source code form. The term was first coined by Richard Stallman, who codified it in the GNU General Public License (GPL), and who founded the Free Software Foundation (http://www.fsf.org/) to promote the concept.

  Although "free software" covers almost exactly the same range of software as "open source", the FSF, among others, prefers the former term because it emphasizes the idea of freedom, and the concept of freely redistributable software as primarily a social movement rather than a technical one. The FSF acknowledges that the term is ambiguous—it could mean "free" as in "zero-cost", instead of "free" as in "freedom"—but feels that it's still the best term, all things considered, and that the other possibilities in English have their own ambiguities. (Throughout this book, "free" is used in the "freedom" sense, not the "zero-cost" sense.)

open source software
: Free software under another name. But the different name reflects an important philosophical difference: "open source" was coined by the Open Source Initiative (http://www.opensource.org/) as a deliberate alternative to "free software," in order to make such software a more palatable choice for corporations, by presenting it as a development methodology rather than a political movement. They may also have wanted to overcome another stigma: that anything "free" must be low quality.

  While any license that is free is also open source, and vice versa (with a few minor exceptions), people tend to pick one term and stick with it. In general, those who prefer "free software" are more likely to have a philosophical or moral stance on the issue, while those who prefer "open source" either don't view it as a matter of freedom, or are not interested in advertising the fact that they do. See the section called “"Free" Versus "Open Source"” in Chapter 1, *Introduction* for a more detailed history of this schism.

  The Free Software Foundation has an excellent—utterly unobjective, but nuanced and quite fair—exegesis of the two terms, at http://www.fsf.org/licensing/essays/free-software-for-freedom.html. The Open Source Initiative's take on it is spread across two pages: http://www.opensource.org/advocacy/case_for_hackers.php#marketing and http://www.opensource.org/advocacy/free-notfree.php.

FOSS, *F/OSS*, *FLOSS*
: Where there are two of anything, there will soon be three, and that is exactly what is happening with terms for free software. The academic world, perhaps wanting precision and inclusiveness over el-

egance, seems to have settled on FOSS, or sometimes F/OSS, standing for "Free / Open Source Software". Another variant gaining momentum is FLOSS, which stands for "Free / Libre Open Source Software" (*libre* is familiar in many languages and does not suffer from the ambiguities of "free"; see http://en.wikipedia.org/wiki/FLOSS for more).

All these terms mean essentially the same thing: software that can be modified and redistributed by everyone, sometimes—but not always—with the requirement that derivative works be freely redistributable under the same terms.

DFSG-compliant
Compliant with the Debian Free Software Guidelines (http://www.debian.org/social_contract#guidelines). This is a widely-used test for whether a given license is truly open source (free, *libre*, etc.). The Debian Project's mission is to maintain an entirely free operating system, such that someone installing it need never doubt that she has the right to modify and redistribute any or all of the system. The Debian Free Software Guidelines are the requirements that a software package's license must meet in order to be included in Debian. Because the Debian Project spent a good deal of time thinking about how to construct such a test, the guidelines they came up with have proven very robust (see http://en.wikipedia.org/wiki/DFSG), and as far as I'm aware, no serious objection to them has been raised either by the Free Software Foundation or the Open Source Initiative. If you know that a given license is DFSG-compliant, you know that it guarantees all the important freedoms (such as forkability even against the original author's wishes) required to sustain the dynamics of an open source project. All of the licenses discussed in this chapter are DFSG-compliant.

OSI-approved
Approved by the Open Source Initiative. This is another widely-used test of whether a license permits all the necessary freedoms. The OSI's definition of open source software is based on the Debian Free Software Guidelines, and any license that meets one definition almost always meets the other. There have been a few exceptions over the years, but only involving niche licenses and none of any relevance here. Unlike the Debian Project, the OSI maintains a list of all licenses it has ever approved, at http://www.opensource.org/licenses/, so that being "OSI-approved" is an unambiguous state: a license either is or isn't on the list.

The Free Software Foundation also maintains a list of licenses at http://www.fsf.org/licensing/licenses/license-list.html. The FSF categorizes licenses not only by whether they are free, but whether they are compatible with the GNU General Public License. GPL compatibility is an important topic, covered in the section called “The GPL and License Compatibility” later in this chapter.

proprietary, *closed-source*
The opposite of "free" or "open source." It means software distributed under traditional, royalty-based licensing terms, where users pay per copy, or under any other terms sufficiently restrictive to prevent open source dynamics from operating. Even software distributed at no charge can still be proprietary, if its license does not permit free redistribution and modification.

Generally "proprietary" and "closed-source" are synonyms. However, "closed-source" additionally implies that the source code cannot even be seen. Since the source code cannot be seen with most proprietary software, this is normally a distinction without a difference. However, occasionally someone releases proprietary software under a license that allows others to view the source code. Confusingly, they sometimes call this "open source" or "nearly open source," etc., but that's misleading. The *visibility* of the source code is not the issue; the important question is what you're allowed to do with it. Thus, the difference between proprietary and closed-source is mostly irrelevant, and the two can be treated as synonyms.

Sometimes *commercial* is used as a synonym for "proprietary," but properly speaking, the two are not the same thing. Free software can be commercial software. After all, free software can be sold, as long as the buyers are not restricted from giving away copies themselves. It can be commercialized in other ways as well, for example by selling support, services, and certification. There are multimillion dollar companies built on free software today, so it is clearly neither inherently anti-

commercial nor anti-corporate. On the other hand, it *is* anti-proprietary by its nature, and this is the key way in which it differs from traditional per-copy license models.

public domain
: Having no copyright holder, meaning that there is no one who has the right to restrict copying of the work. Being in the public domain is not the same as having no author. Everything has an author, and even if a work's author or authors choose to put it in the public domain, that doesn't change the fact that they wrote it.

  When a work is in the public domain, material from it can be incorporated into a copyrighted work, and thereafter *that copy* of the material is covered under the same copyright as the whole work. But this does not affect the availability of the original work, which remains in the public domain. Thus, releasing something into the public domain is technically one way to make it "free," according to the guidelines of most free software certifying organizations. However, there are usually good reasons to use a license instead of just releasing into the public domain: even with free software, certain restrictions can be useful, not only to the copyright holder but even to recipients as well, as the next section makes clear.

copyleft
: A license that uses copyright law to achieve a result opposite to traditional copyright. Depending on whom you ask, this means either licenses that permit the freedoms under discussion here, or, more narrowly, licenses that not only permit those freedoms but *enforce* them, by stipulating that the freedoms must travel with the work. The Free Software Foundation uses the second definition exclusively; elsewhere, it's a toss-up: a lot of people use the term the same way the FSF does, but others—including some who write for mainstream media—tend to use the first definition. It's not clear that everyone using the term is aware that there's a distinction to be made.

  The canonical example of the narrower, stricter definition is the GNU General Public License, which stipulates that any derivative works must also be licensed under the GPL; see the section called “The GPL and License Compatibility” later in this chapter for more.

Aspects of Licenses

Although there are many different free software licenses available, in the important respects they all say the same things: that anyone can modify the code, that anyone can redistribute it both in original and modified form, and that the copyright holders and authors provide no warranties whatsoever (avoiding liability is especially important given that people might run modified versions without even knowing it). The differences between licences boil down to a few oft-recurring issues:

compatibility with proprietary licenses
: Some free licenses allow the covered code to be used in proprietary programs. This does not affect the licensing terms of the proprietary program: it is still as proprietary as ever, it just happens to contain some code from a non-proprietary source. The Apache License, X Consortium License, BSD-style license, and the MIT-style license are all examples of proprietary-compatible licenses.

compatibility with other free licenses
: Most free licenses are compatible with each other, meaning that code under one license can be combined with code under another, and the result distributed under either license without violating the terms of the other. The major exception to this is the GNU General Public License, which requires that any work using GPLed code be itself distributed under the GPL, and without adding any further restrictions beyond what the GPL requires. The GPL is compatible with some free licenses, but not with others. This is discussed in more detail in the section called “The GPL and License Compatibility” later in this chapter.

enforcement of crediting
: Some free licenses stipulate that any use of the covered code be accompanied by a notice, whose

placement and display is usually specified, giving credit to the authors or copyright holders of the code. These licenses are often still proprietary-compatible: they do not necessarily demand that the derivative work be free, merely that credit be given to the free code.

protection of trademark
: A variant of credit enforcement. Trademark-protecting licenses specify that the name of the original software (or its copyright holders, or their institution, etc.) may *not* be used by derivative works without prior written permission. Although credit enforcement insists that a certain name be used, and trademark protection insists that it not be used, they are both expressions of the same desire: that the original code's reputation be preserved and transmitted, but not tarnished by association.

protection of "artistic integrity"
: Some licenses (the Artistic License, used for the most popular implementation of the Perl programming language, and Donald Knuth's TeX license, for example) require that modification and redistribution be done in a manner that distinguishes clearly between the pristine original version of the code and any modifications. They permit essentially the same freedoms as other free license, but impose certain requirements that make the integrity of the original code easy to verify. These licenses have not caught on much beyond the specific programs they were made for, and will not be discussed in this chapter; they are mentioned here only for the sake of completeness.

Most of these stipulations are not mutually exclusive, and some licenses include several. The common thread among them is that they place demands on the recipient in exchange for the recipient's right to use and/or redistribute the code. For example, some projects want their name and reputation to be transmitted along with the code, and this is worth imposing the extra burden of a credit or trademark clause; depending on its onerousness, that burden may result in some users choosing a package with a less demanding license.

The GPL and License Compatibility

By far the sharpest dividing line in licensing is that between proprietary-incompatible and proprietary-compatible licenses, that is, between the GNU General Public License and everything else. Because the primary goal of the GPL's authors is the promotion of free software, they deliberately crafted the license to make it impossible to mix GPLed code into proprietary programs. Specifically, among the GPL's requirements (see http://www.fsf.org/licensing/licenses/gpl.html for its full text) are these two:

1. Any derivative work—that is, any work containing a nontrivial amount of GPLed code—must itself be distributed under the GPL.

2. No additional restrictions may be placed on the redistribution of either the original work or a derivative work. (The exact language is: "You may not impose any further restrictions on the recipients' exercise of the rights granted herein.")

With these conditions, the GPL succeeds in making freedom contagious. Once a program is copyrighted under the GPL, its terms of redistribution are *viral*—they are passed on to anything else the code gets incorporated into, making it effectively impossible to use GPLed code in closed-source programs. However, these same clauses also make the GPL incompatible with certain other free licenses. The usual way this happens is that the other license imposes a requirement—for example, a credit clause requiring the original authors to be mentioned in some way—that is incompatible with the GPL's "You may not impose any further restrictions..." language. From the point of view of the Free Software Foundation, these second-order consequences are desirable, or at least not regrettable. The GPL not only keeps your software free, but effectively makes your software an agent in pushing *other* software to enforce freedom as well.

The question of whether or not this is a good way to promote free software is one of the most persistent holy wars on the Internet (see the section called “Avoid Holy Wars” in Chapter 6, *Communications*),

and we won't investigate it here. What's important for our purposes is that GPL compatibility is an important issue when choosing a license. The GPL is by far the most popular open source license; at http://freshmeat.net/stats/#license, it is at 68%, and the next highest license is at 6%. If you want your code to be able to be mixed freely with GPLed code—and there's a lot of GPLed code out there—then you should pick a GPL-compatible license. Most of the GPL-compatible open source licenses are also proprietary-compatible: that is, code under such a license can be used in a GPLed program, and it can be used in a proprietary program. Of course, the *results* of these mixings would not be compatible with each other, since one would be under the GPL and the other would be under a closed-source license. But that concern applies only to the derivative works, not to the code you distribute in the first place.

Fortunately, the Free Software Foundation maintains a list showing which licenses are compatible with the GPL and which are not, at http://www.gnu.org/licenses/license-list.html. All of the licenses discussed in this chapter are present on that list, on one side or the other.

Choosing a License

When choosing a license to apply to your project, if at all possible use an existing license instead of making up a new one. There are two reasons why existing licenses are better:

- Familiarity. If you use one of the three or four most popular licenses, people won't feel they have to read the legalese in order to use your code, because they'll have already done so for that license a long time ago.
- Quality. Unless you have a team of lawyers at your disposal, you are unlikely to come up with a legally solid license. The licenses mentioned here are the products of much thought and experience; unless your project has truly unusual needs, it is unlikely you would do better.

To apply one of these licenses to your project, see the section called “How to Apply a License to Your Software” in Chapter 2, *Getting Started*.

The MIT / X Window System License

If your goal is that your code be accessible by the greatest possible number of developers and derivative works, and you do not mind the code being used in proprietary programs, choose the MIT / X Window System license (so named because it is the license under which the Massachusetts Institute of Technology released the original X Window System code). This license's basic message is "You are free to use this code however you want." It is compatible with the GNU GPL, and it is short, simple, and easy to understand:

```
Copyright (c) <year> <copyright holders>

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE
LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION
```

```
OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION
WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

(Taken from http://www.opensource.org/licenses/mit-license.php.)

The GNU General Public License

If you prefer that your project's code not be used in proprietary programs, or if you at least don't care whether or not it can be used in proprietary programs, choose the GNU General Public License (http://www.fsf.org/licensing/licenses/gpl.html). The GPL is probably the most widely-used free software license in the world today; this instant recognizability is itself one of the GPL's major advantages.

When writing a code library that is meant mainly to be used as part of other programs, consider carefully whether the restrictions imposed by the GPL are in line with your project's goals. In some cases—for example, when you're trying to unseat a competing, proprietary library that does the same thing—it may make more strategic sense to license your code in such a way that it can be mixed into proprietary programs, even though you would otherwise not wish this. The Free Software Foundation even fashioned an alternative to the GPL for such circumstances: the *GNU Library GPL*, later renamed to the *GNU Lesser GPL* (most people just use the acronym *LGPL*, in any case). The LGPL has looser restrictions than the GPL, and can be mixed more easily with non-free code. However, it's also a bit complex and takes some time to understand, so if you're not going to use the GPL, I recommend just using the MIT/X-style license.

Is the GPL free or not free?

One consequence of choosing the GPL is the possibility—small, but not infinitely small—of finding yourself or your project embroiled in a dispute about whether or not the GPL is truly "free", given that it places some restrictions on what you can do with the code—namely, the restriction that the code cannot be distributed under any other license. For some people, the existence of this restriction means the GPL is "less free" than more permissive licenses such as the MIT/X license. Where this argument usually goes, of course, is that since "more free" must be better than "less free" (after all, who's not in favor of freedom?), it follows that those licenses are better than the GPL.

This debate is another popular holy war (see the section called “Avoid Holy Wars” in Chapter 6, *Communications*). Avoid participating in it, at least in project forums. Don't attempt to prove that the GPL is less free, as free, or more free than other licenses. Instead, emphasize the specific reasons your project chose the GPL. If the recognizability of license was a reason, say that. If the enforcement of a free license on derivative works was also a reason, say that too, but refuse to be drawn into discussion about whether this makes the code more or less "free". Freedom is a complex topic, and there is little point talking about it if terminology is going to be used as a stalking horse for substance.

Since this is a book and not a mailing list thread, however, I will admit that I've never understood the "GPL is not free" argument. The only restriction the GPL imposes is that it prevents people from imposing *further* restrictions. To say that this results in less freedom has always seemed to me like saying that outlawing slavery reduces freedom, because it prevents some people from owning slaves.

(Oh, and if you do get drawn into a debate about it, don't raise the stakes by making inflammatory analogies.)

What About The BSD License?

A fair amount of open source software is distributed under a *BSD license* (or sometimes a *BSD-style license*). The original BSD license was used for the Berkeley Software Distribution, in which the University of California released important portions of a Unix implementation. This license (the exact text may be seen in section 2.2.2 of http://www.xfree86.org/3.3.6/COPYRIGHT2.html#6) was similar in spirit to the MIT/X license, except for one clause:

> *All advertising materials mentioning features or use of this software must display the following acknowledgement: This product includes software developed by the University of California, Lawrence Berkeley Laboratory.*

The presence of that clause not only made the original BSD license GPL-incompatible, it also set a dangerous precedent: as other organizations put similar advertising clauses into *their* free software—substituting their own organization's name in place of "the University of California, Lawrence Berkeley Laboratory"—software redistributors faced an ever-increasing burden in what they were required to display. Fortunately, many of the projects that used this license became aware of the problem, and simply dropped the advertising clause. In 1999, even the University of California did so.

The result is the revised BSD license, which is simply the original BSD license with the advertising clause removed. However, this history makes the phrase "BSD license" a bit ambiguous: does it refer to the original, or the revised version? This is why I prefer the MIT/X license, which is essentially equivalent, and which does not suffer from any ambiguity. However, there is perhaps one reason to prefer the revised BSD license to the MIT/X license, which is that the BSD includes this clause:

> *Neither the name of the <ORGANIZATION> nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.*

It's not clear that without such a clause, a recipient of the software would have had the right to use the licensor's name anyway, but the clause removes any possible doubt. For organizations worried about trademark control, therefore, the revised BSD license may be slightly preferable to MIT/X. In general, however, a liberal copyright license does not imply that recipients have any right to use or dilute your trademarks — copyright law and trademark law are two different beasts.

If you wish to use the revised BSD license, a template is available at http://www.opensource.org/licenses/bsd-license.php.

Copyright Assignment and Ownership

There are three ways to handle copyright ownership for free code and documentation that were contributed to by many people. The first is to ignore the issue of copyright entirely (I don't recommend this). The second is to collect a *contributor license agreement* (*CLA*) from each person who works on the project, explicitly granting the project the right to use that person's contributions. This is usually enough for most projects, and the nice thing is that in some jurisdictions, CLAs can be sent in by email. The third way is to get actual copyright assignments from contributors, so that the project (i.e., some legal entity, usually a nonprofit) is the copyright owner for everything. This is the most legally airtight way, but it's also the most burdensome for contributors; only a few projects insist on it.

Note that even under centralized copyright ownership, the code[1] remains free, because open source licenses do not give the copyright holder the right to retroactively proprietize all copies of the code. So even if the project, as a legal entity, were to suddenly turn around and started distributing all the code under a restrictive license, that wouldn't cause a problem for the public community. The other developers would simply start a fork based on the latest free copy of the code, and continue as if nothing had happened. Because they know they can do this, most contributors cooperate when asked to sign a CLA or an assignment of copyright.

Doing Nothing

Most projects never collect CLAs or copyright assignments from their contributors. Instead, they accept code whenever it seems reasonably clear that the contributor intended it to be incorporated into the

[1] I'll use "code" to refer to both code and documentation, from now on.

project.

Under normal circumstances, this is okay. But every now and then, someone may decide to sue for copyright infringement, alleging that they are the true owner of the code in question and that they never agreed to its being distributed by the project under an open source license. For example, the SCO Group did something like this to the Linux project, see http://en.wikipedia.org/wiki/SCO-Linux_controversies for details. When this happens, the project will have no documentation showing that the contributor formally granted the right to use the code, which could make some legal defenses more difficult.

Contributor License Agreements

CLAs probably offer the best tradeoff between safety and convenience. A CLA is typically an electronic form that a developer fills out and sends in to the project. In many jurisdictions, email submission is enough. A secure digital signature may or may not be required; consult a lawyer to find out what method would be best for your project.

Most projects use two slightly different CLAs, one for individuals, and one for corporate contributors. But in both types, the core language is the same: the contributor grants the project *"...perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare derivative works of, publicly display, publicly perform, sublicense, and distribute [the] Contributions and such derivative works."* Again, you should have a lawyer approve any CLA, but if you get all those adjectives into it, you're probably fine.

When you request CLAs from contributors, make sure to emphasize that you are *not* asking for actual copyright assignment. In fact, many CLAs start out by reminding the reader of this:

> *This is a license agreement only; it does not transfer copyright ownership and does not change your rights to use your own Contributions for any other purpose.*

Here are some examples:

- Individual contributor CLAs:
  - http://apache.org/licenses/icla.txt
  - http://code.google.com/legal/individual-cla-v1.0.html
- Corporate contributor CLAs:
  - http://apache.org/licenses/cla-corporate.txt
  - http://code.google.com/legal/corporate-cla-v1.0.html

Transfer of Copyright

Copyright transfer means that the contributor assigns to the project copyright ownership on her contributions. Ideally, this is done on paper and either faxed or snail-mailed to the project.

Some projects insist on full assignment because having a single legal entity own the copyright on the entire code base can be useful if the terms of the open source license ever need to be enforced in court. If no single entity has the right to do it, all the contributors might have to cooperate, but some might not have time or even be reachable when the issue arises.

Different organizations apply different amounts of rigor to the task of collecting assignments. Some simply get an informal statement from a contributor on a public list mailing list—something to the effect

of "I hereby assign copyright in this code to the project, to be licensed under the same terms as the rest of the code." At least one lawyer I've talked to says that's really enough, presumably because it happens in a context where copyright assignment is normal and expected anyway, and because it represents a *bona fide* effort on the project's part to ascertain the developer's true intentions. On the other hand, the Free Software Foundation goes to the opposite extreme: they require contributors to physically sign and mail in a piece of paper containing a formal statement of copyright assignment, sometimes for just one contribution, sometimes for current and future contributions. If the developer is employed, the FSF asks that the employer sign it too.

The FSF's paranoia is understandable. If someone violates the terms of the GPL by incorporating some of their software into a proprietary program, the FSF will need to fight that in court, and they want their copyrights to be as airtight as possible when that happens. Since the FSF is copyright holder for a lot of popular software, they view this as a real possibility. Whether your organization needs to be similarly scrupulous is something only you can decide, in consultation with lawyers. In general, unless there's some specific reason why your project needs full copyright assignment, just go with CLAs; they're easier for everyone.

Dual Licensing Schemes

Some projects try to fund themselves by using a dual licensing scheme, in which proprietary derivative works may pay the copyright holder for the right to use the code, but the code still remains free for use by open source projects. This tends to work better with code libraries than with standalone applications, naturally. The exact terms differ from case to case. Often the license for the free side is the GNU GPL, since it already bars others from incorporating the covered code into their proprietary product without permission from the copyright holder, but sometimes it is a custom license that has the same effect. An example of the former is the MySQL license, described at http://www.mysql.com/company/legal/licensing/; an example of the latter is Sleepycat Software's licensing strategy, described at http://www.oracle.com/technology/software/products/berkeley-db/htdocs/licensing.html.

You might be wondering: how can the copyright holder offer proprietary licensing for a mandatory fee if the terms of the GNU GPL stipulate that the code must be available under less restrictive terms? The answer is that the GPL's terms are something the copyright holder imposes on everyone else; the owner is therefore free to decide *not* to apply those terms to itself. A good way to think of it is to imagine that the copyright owner has an infinite number of copies of the software stored in a bucket. Each time it takes one out of the bucket to send into the world, it can decide what license to put on it: GPL, proprietary, or something else. Its right to do this is not tied to the GPL or any other open source license; it is simply a power granted by copyright law.

The attractiveness of dual licensing is that, at its best, it provides a way for a free software project to get a reliable income stream. Unfortunately, it can also interfere with the normal dynamics of open source projects. The problem is that any volunteer who makes a code contribution is now contributing to two distinct entities: the free version of the code and the proprietary version. While the contributor will be comfortable contributing to the free version, since that's the norm in open source projects, she may feel funny about contributing to someone else's semi-proprietary revenue stream. The awkwardness is exacerbated by the fact that in dual licensing, the copyright owner really needs to gather formal, signed copyright assignments from all contributors, in order to protect itself from a disgruntled contributor later claiming a percentage of royalties from the proprietary stream. The process of collecting these assignment papers means that contributors are starkly confronted with the fact that they are doing work that makes money for someone else.

Not all volunteers will be bothered by this; after all, their contributions go into the open source edition as well, and that may be where their main interest lies. Nevertheless, dual licensing is an instance of the copyright holder assigning itself a special right that others in the project do not have, and is thus bound to raise tensions at some point, at least with some volunteers.

What seems to happen in practice is that companies based on dual licensed software do not have truly

egalitarian development communities. They get small-scale bug fixes and cleanup patches from external sources, but end up doing most of the hard work with internal resources. For example, Zack Urlocker, vice president of marketing at MySQL, told me that the company generally ends up hiring the most active volunteers anyway. Thus, although the product itself is open source, licensed under the GPL, its development is more or less controlled by the company, albeit with the (extremely unlikely) possibility that someone truly dissatisfied with the company's handling of the software could fork the project. To what degree this threat preëmptively shapes the company's policies I don't know, but at any rate, MySQL does not seem to be having acceptance problems either in the open source world or beyond.

Patents

Software patents are the lightning rod issue of the moment in free software, because they pose the only real threat against which the free software community cannot defend itself. Copyright and trademark problems can always be gotten around. If part of your code looks like it may infringe on someone else's copyright, you can just rewrite that part. If it turns out someone has a trademark on your project's name, at the very worst you can just rename the project. Although changing names would be a temporary inconvenience, it wouldn't matter in the long run, since the code itself would still do what it always did.

But a patent is a blanket injunction against implementing a certain idea. It doesn't matter who writes the code, nor even what programming language is used. Once someone has accused a free software project of infringing a patent, the project must either stop implementing that particular feature, or face an expensive and time-consuming lawsuit. Since the instigators of such lawsuits are usually corporations with deep pockets—that's who has the resources and inclination to acquire patents in the first place—most free software projects cannot afford the latter possibility, and must capitulate immediately even if they think it highly likely that the patent would be unenforceable in court. To avoid getting into such a situation in the first place, free software projects are starting to code defensively, avoiding patented algorithms in advance even when they are the best or only available solution to a programming problem.[2]

Surveys and anecdotal evidence show that not only the vast majority of open source programmers, but a majority of *all* programmers, think that software patents should be abolished entirely.[3] Open source programmers tend to feel particularly strongly about it, and may refuse to work on projects that are too closely associated with the collection or enforcement of software patents. If your organization collects software patents, then make it clear, in a public and irrevocable way, that the patents would never be enforced on open source projects, and that they are only to be used as a defense in case some other party initiates an infringement suit against your organization. This is not only the right thing to do, it's also good open source public relations.[4]

Unfortunately, collecting patents for defensive purposes is a rational action. The current patent system, at least in the United States, is by its nature an arms race: if your competitors have acquired a lot of patents, then your best defense is to acquire a lot of patents yourself, so that if you're ever hit with a patent infringement suit you can respond with a similar threat—then the two parties usually sit down and work out a cross-licensing deal so that neither of them has to pay anything, except to their intellectual property lawyers of course.

The harm done to free software by software patents is more insidious than just direct threats to code development, however. Software patents encourage an atmosphere of secrecy among firmware designers, who justifiably worry that by publishing details of their interfaces they will be giving technical help to competitors seeking to slap them with patent infringement suits. This is not just a theoretical danger; it has apparently been happening for a long time in the video card industry, for example. Many video card manufacturers are reluctant to release the detailed programming specifications needed to produce high-

[2]Sun Microsystems and IBM have also made at least a gesture at the problem from the other direction, by freeing large numbers of software patents—1600 and 500 respectively—for use by the open source community. I am not a lawyer and thus can't evaluate the real utility of these grants, but even if they are all important patents, and the terms of the grants make them truly free for use by any open source project, it would still be only a drop in the bucket.

[3]See http://lpf.ai.mit.edu/Whatsnew/survey.html for one such survey.

[4]For example, RedHat has pledged that open source projects are safe from its patents, see http://www.redhat.com/legal/patent_policy.html.

performance open source drivers for their cards, thus making it impossible for free operating systems to support those cards to their full potential. Why would the manufacturers do this? It doesn't make sense for them to work *against* software support; after all, compatibility with more operating systems can only mean more card sales. But it turns out that, behind the design room door, these shops are all violating one another's patents, sometimes knowingly and sometimes accidentally. The patents are so unpredictable and so potentially broad that no card manufacturer can ever be certain it's safe, even after doing a patent search. Thus, manufacturers dare not publish their full interface specifications, since that would make it much easier for competitors to figure out whether any patents are being infringed. (Of course, the nature of this situation is such that you will not find a written admission from a primary source that it is going on; I learned it through a personal communication.)

Some free software licenses have special clauses to combat, or at least discourage, software patents. The GNU GPL, for example, contains this language:

```
  7. If, as a consequence of a court judgment or allegation of patent
infringement or for any other reason (not limited to patent issues),
conditions are imposed on you (whether by court order, agreement or
otherwise) that contradict the conditions of this License, they do not
excuse you from the conditions of this License.  If you cannot
distribute so as to satisfy simultaneously your obligations under this
License and any other pertinent obligations, then as a consequence you
may not distribute the Program at all.  For example, if a patent
license would not permit royalty-free redistribution of the Program by
all those who receive copies directly or indirectly through you, then
the only way you could satisfy both it and this License would be to
refrain entirely from distribution of the Program.

[...]

It is not the purpose of this section to induce you to infringe any
patents or other property right claims or to contest validity of any
such claims; this section has the sole purpose of protecting the
integrity of the free software distribution system, which is
implemented by public license practices.  Many people have made
generous contributions to the wide range of software distributed
through that system in reliance on consistent application of that
system; it is up to the author/donor to decide if he or she is willing
to distribute software through any other system and a licensee cannot
impose that choice.
```

The Apache License, Version 2.0 (http://www.apache.org/licenses/LICENSE-2.0) also contains anti-patent requirements. First, it stipulates that anyone distributing code under the license must implicitly include a royalty-free patent license for any patents they might hold that could apply to the code. Second, and most ingeniously, it punishes anyone who initiates a patent infringement claim on the covered work, by automatically terminating their implicit patent license the moment such a claim is made:

```
3. Grant of Patent License. Subject to the terms and conditions of
this License, each Contributor hereby grants to You a perpetual,
worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except
as stated in this section) patent license to make, have made, use,
offer to sell, sell, import, and otherwise transfer the Work, where
such license applies only to those patent claims licensable by such
Contributor that are necessarily infringed by their Contribution(s)
alone or by combination of their Contribution(s) with the Work to
which such Contribution(s) was submitted. If You institute patent
litigation against any entity (including a cross-claim or counterclaim
in a lawsuit) alleging that the Work or a Contribution incorporated
within the Work constitutes direct or contributory patent
infringement, then any patent licenses granted to You under this
```

```
License for that Work shall terminate as of the date such litigation
is filed.
```

Although it is useful, both legally and politically, to build patent defenses into free software licenses this way, in the end these steps will not be enough to dispel the chilling effect that the threat of patent lawsuits has on free software. Only changes in the substance or interpretation of international patent law will do that. To learn more about the problem, and how it's being fought, go to http://www.nosoftwarepatents.com/. The Wikipedia article http://en.wikipedia.org/wiki/Software_patent also has a lot of useful information on software patents. I've also written a blog post summarizing the arguments against software patents, at http://www.rants.org/2007/05/01/how-to-tell-that-software-patents-are-a-bad-idea/.

Further Resources

This chapter has only been an introduction to free software licensing issues. Although I hope it contains enough information to get you started on your own open source project, any serious investigation of licensing issues will quickly exhaust what this book can provide. Here is a list of further resources on open source licensing:

- *Understanding Open Source and Free Software Licensing* by Andrew M. St. Laurent. Published by O'Reilly Media, first edition August 2004, ISBN: 0-596-00581-4.

  This is a full-length book on open source licensing in all its complexity, including many topics omitted from this chapter. See http://www.oreilly.com/catalog/osfreesoft/ for details.

- *Make Your Open Source Software GPL-Compatible. Or Else.* by David A. Wheeler, at http://www.dwheeler.com/essays/gpl-compatible.html.

  This is a detailed and well-written article on why it is important to use a GPL-compatible license even if you don't use the GPL itself. The article also touches on many other licensing questions, and has a high density of excellent links.

- http://creativecommons.org/

  Creative Commons is an organization that promotes a range of more flexible and liberal copyrights than traditional copyright practice encourages. They offer licenses not just for software, but for text, art, and music as well, all accessible via a user-friendly license selector; some of the licenses are copylefts, some are non-copyleft but still free, others are simply traditional copyrights but with some restrictions relaxed. The Creative Commons web site gives extremely clear explanations of what it's about. If I had to pick one site to demonstrate the broader philosophical implications of the free software movement, this would be it.